

字典树Trie

1.基本操作

```
1  const int maxn=100010;//代表容量;
2  const int charsize=26;//字符集大小
3  int root=0,cnt=0;
4  int nxt[maxn+1][charsize];//nxt[i]['b'-'a']表示i号节点的子节点中代表字母b的节点编
   号;
5  bool isend[maxn+1];
6
7  void insert(char s[],int len)
8  {
9      int now=0;
10     for(int i=1;i<=len;i++)
11     {
12         int tempt=s[i]-'a';
13         if(!nxt[now][tempt])
14             nxt[now][tempt]=++cnt;//用cnt来记录一共有多少个字符加入这个树，从而编
   号;
15         now=nxt[now][x];//继续填下一个;
16     }
17     isend[now]=1;//标记字符串结尾;
18 }
19
20 bool search(char s[],int len)
21 {
22     int now=0;
23     for(int i=1;i<=len;i++)
24     {
25         int tempt=s[i]-'a';
26         if(!nxt[now][x])
27             return false;
28         now=nxt[now][x];
29     }
30     return isend[now];//判断是否有以它结尾的;
31 }
```

2.例题

1)



- 给你n个字符串，接下来有m个询问，每次询问一个字符串是几个字符串的前缀。
- 字符串的前缀是指从这个串首开始到某个位置结束的，前i个字母组成的子串。
- 输出共m行，每行1个数表示答案。

• 样例输入：

```
4
abcd
cdf
abc
bbc
2
ab
cd
```

• 样例输出：

```
2
1
```

- 数据范围： $1 \leq n, m \leq 50000$, $1 \leq \text{所有字符串长度} \leq 10$, 字符串均由小写字母构成。

```
1  #include<iostream>
2  using namespace std;
3  const int maxn = 500001;
4  const int charsize = 26;
5  int nxt[maxn + 1][charsize];
6  int cnt = 0;
7  int tot[maxn + 1];
8  void Insert(char s[], int len)
9  {
10     int now = 0;
11     for (int i = 0; i < len; i++)
12     {
13         int tempt = s[i] - 'a';
14         if (!nxt[now][tempt])
15             nxt[now][tempt] = ++cnt;
16         now = nxt[now][tempt];
17         tot[now]++;
18     }
19 }
20 int Search(char s[], int len)
21 {
22     int now = 0;
23     for (int i = 0; i < len; i++)
24     {
25         int tempt = s[i] - 'a';
26         if (!nxt[now][tempt])
27             return -1;
28         now = nxt[now][tempt];
29     }
30     return now;
31 }
32 int main()
33 {
34     int n;
35     cin >> n;
```

```

36     for (int i = 1; i <= n; i++)
37     {
38         char str[11];
39         cin >> str;
40         int len = strlen(str);
41         Insert(str, len);
42     }
43     int m;
44     cin >> m;
45     for (int i = 1; i <= m; i++)
46     {
47         char find[11];
48         cin >> find;
49         int len = strlen(find);
50         int end=Search(find, len);
51         if (end == -1)
52             cout << 0;
53         else
54             cout << tot[end];
55     }
56 }
57

```

这道题最关键的是tot数组，在创建字典树的时候，每有一个字符串经过某个节点，tot[该节点]++; 这样就很方便统计下面连接了几个字符串；（不等价于子树哦）

2)



字符串排序

知乎账号：walker

- 给你n个两两不同字符串，你要按字典序从小到大的顺序将这些字符串排好，再按顺序输出。
- 字符串的字典序是指以第i个字符作为第i关键字进行大小比较的排序，空字符小于字符集内任何字符。

• 样例输入：

```

4
abcd
cdf
abc
bbc

```

• 样例输出：

```

abc
abcd
bbc
cdf

```

- 数据范围：1 ≤ n ≤ 50000, 1 ≤ 所有字符串长度 ≤ 10, 字符串均由小写字母构成。

```

1  #include<iostream>
2  using namespace std;
3  int nxt[1000001][26];
4  bool isend[1000001];
5  int cnt;
6  int a[100001];
7  void DFS(int now, int depth)
8  {

```

```

9      if (isend[now])//说明可以输出一次；但不代表不继续往下搜索；
10     {
11         for (int i = 0; i < depth; i++)
12             cout << (char)(a[i] + 'a');
13         cout << endl;
14     }
15     for (int i = 0; i < 26; i++)
16     {
17         if (nxt[now][i])
18         {
19             a[depth] = i;//下一次的i会冲掉上一次的a[depth]，因此不必回溯
20             DFS(nxt[now][i], depth + 1);
21         }
22     }
23 }
24 int main()
25 {
26     int n;
27     cin >> n;
28     for (int i = 1; i <= n; i++)
29     {
30         char str[20];
31         cin >> str;
32         int len = strlen(str);
33         int now = 0;
34         for (int i = 0; i < len; i++)
35         {
36             int tempt = str[i] - 'a';
37             if (!nxt[now][tempt])
38                 nxt[now][tempt] = ++cnt;
39             now = nxt[now][tempt];
40         }
41         isend[now] = 1;
42     }
43     DFS(0, 0);
44     return 0;
45 }

```

3)



*最长公共前缀问题

知乎账号: walk

- 给你 n 个字符串，接下来有 m 个询问，每次询问第 i 个字符串与第 j 个字符串的最长公共前缀长度。
- 两个字符串 a, b 的最长公共前缀是指：找到最大的长度 k ， a 和 b 前 k 个位置的字符完全一样。
- 输出共 m 行，每行 1 个数表示答案。

• 样例输入：

```
4
abcd
cdf
abc
bbc
2
1 3
2 4
```

• 样例输出：

```
3
0
```

- 数据范围： $1 \leq n, m \leq 50000, 1 \leq i, j \leq n, i \neq j, 1 \leq \text{单个字符串长度} \leq 10000, 1 \leq \text{所有字符串长度} \leq 500000$ ，字符串均由小写字母构成。

知识储备： 倍增法求LCA

```
1  #include<iostream>
2  using namespace std;
3  int cnt;
4  int fa[10001][21]; //记录每个节点的父亲;
5  int strend[1001]; //记录每个字符串的结束的编号;
6  int nxt[10001][26];
7  int dist[10001];
8  int main()
9  {
10     int n;
11     cin >> n;
12     for (int k = 1; k <= n; k++)
13     {
14         int now = 0;
15         char str[20];
16         cin >> str;
17         int len = strlen(str);
18         for (int i = 0; i < len; i++)
19         {
20             int tempt = str[i] - 'a';
21             if (!nxt[now][tempt])
22             {
23                 nxt[now][tempt] = ++cnt;
24                 fa[cnt][0] = now;
25                 dist[cnt] = dist[now] + 1;
26             }
27             now = nxt[now][tempt]; //不可改为cnt, 不是所有情况都进入if;
28         }
29         strend[k] = now;
30     }
31     for (int i = 1; i <= 20; i++)
32     {
33         for (int j = 1; j <= cnt; j++)
34         {
```

```

35         if (fa[j][i - 1])
36             fa[j][i] = fa[fa[j][i - 1]][i - 1];
37     }
38 }
39 int m;
40 cin >> m;
41 for (int i = 1; i <= m; i++)
42 {
43     int x, y;
44     cin >> x >> y;
45     x = strend[x];
46     y = strend[y];
47     if (dist[x] < dist[y])
48         swap(x, y);
49     int z = dist[x] - dist[y];
50     for (int j = 0; j <= 20 && z; z >>= 1)
51     {
52         if (z & 1)
53             x = fa[x][j];
54     }
55     if (x == y)
56     {
57         cout << dist[x]<<endl;
58         continue;
59     }
60     for (int j = 20; j >= 0; j--)
61         if (fa[x][j] != fa[y][j])
62             x = fa[x][j], y = fa[y][j];
63     cout << dist[fa[x][0]]<<endl;
64 }
65 }

```

这道题我们采用了LCA的做法，因此与一般的字典树的初始化有区别；

- 我们打算从字符串的结尾往上搜索公共祖先，因此用一个数组strend来记录每一个字符串的结尾；
- 为了实现往上跳，必须有father数组来记录父亲；
- 为了知道深度，要用dist数组来记录；